

第 7 章：Policy Gradient、REINFORCE（经典 Monte Carlo Policy Gradient 算法）与 Advantage

本章符号说明

符号/缩写	English	中文	含义
$\pi_\theta(a \mid s)$	Parameterized Policy	参数化策略	参数为 θ 的策略分布。
θ	Policy Parameters	策略参数	policy network 的可学习参数。
τ	Trajectory	轨迹	一整段 state-action-reward 序列。
$J(\theta)$	Objective Function	目标函数	policy gradient 中要最大化的期望 return。
$R(\tau)$	Trajectory Return	轨迹回报	整条轨迹的累计 reward。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 开始的累计收益。
$b(s)$	Baseline	基线函数	用来降低 policy gradient 方差的 action-independent 函数。
$V_\pi(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_\pi(s,a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s,a) 的长期价值。
$A_\pi(s,a)$	Advantage Function	优势函数	$Q_\pi(s,a) - V_\pi(s)$ ，表示动作比平均水平好多少。
$H(\pi)$	Entropy	熵	衡量策略随机性，常用于鼓励探索。
RL	Reinforcement Learning	强化学习	Policy Gradient 是 RL 中直接优化策略的一类方法。
SAC	Soft Actor-Critic	软 Actor-Critic	后续最大熵 actor-critic 算法，本章先介绍 entropy 思想。

本章目标

本章学习 policy-based RL 的基础。你需要掌握：

- 为什么需要直接优化 policy。

- stochastic policy。
- policy gradient theorem。
- log-derivative trick。
- REINFORCE。
- baseline。
- advantage function。

本章主线

Value-based 方法先学 $Q(s,a)$ ，再通过 $\operatorname{argmax}_a Q(s,a)$ 选动作。这个思路在连续动作、随机策略或需要直接控制动作分布时不够自然。本章转向 policy-based RL：直接参数化 $\pi_\theta(a/s)$ ，推导如何对期望 return 求梯度，并用 baseline 和 advantage 降低方差。

本章新增概念说明

本章从 value-based 转到 policy-based，先定义这些词：

名词	中文	先记住什么
value-based	基于价值的方法	先学 $Q(s,a)$ 或 $V(s)$ ，再用价值选动作。
policy-based	基于策略的方法	直接学习 $\pi_\theta(a/s)$ ，让策略本身输出动作或动作分布。
stochastic policy	随机策略	输出动作概率分布，采样动作；天然带探索。
deterministic policy	确定性策略	给定状态直接输出一个动作。
categorical distribution	类别分布	离散动作空间常用的动作概率分布。
Gaussian policy	高斯策略	连续动作空间常用的动作分布，输出均值和方差。
trajectory	轨迹	一整段 state-action-reward 序列。
log-derivative trick	对数导数技巧	把 ∇p 改写成 $p \nabla \log p$ ，用于对采样分布求梯度。
policy gradient theorem	策略梯度定理	说明如何对期望 return 关于策略参数求梯度。
REINFORCE	REINFORCE 算法	最基础的 Monte Carlo policy gradient 算法。
reward-to-go	从当前步开始的回报	只用当前动作之后的 reward 更新当前动作，降低方差。
baseline	基线	从 return 中减掉的 action-independent 参考值，用来降方差。

名词	中文	先记住什么
advantage	优势函数	动作比该状态平均水平好多少, 即 $Q(s,a)-V(s)$ 。
entropy regularization	熵正则	鼓励策略保持随机性, 避免过早确定。

为什么需要 Policy Gradient

Value-based 方法学习 $Q(s,a)$, 再通过 $\operatorname{argmax}_a Q(s,a)$ 选动作。它在离散动作空间中很自然, 但在以下场景中不够方便:

- 动作空间连续。
- 策略本身需要随机性。
- 希望直接优化策略分布。
- $\operatorname{argmax}_a Q(s,a)$ 很难计算。

Policy Gradient 直接参数化策略:

$$\pi_{\theta}(a \mid s)$$

目标是最大化期望 return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

其中 trajectory:

$$\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$$

随机策略

离散动作空间中, 策略可以是 categorical distribution:

$$\pi_{\theta}(a \mid s) = \operatorname{softmax}(f_{\theta}(s))$$

连续动作空间中, 策略常用 Gaussian:

$$\pi_{\theta}(a \mid s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s))$$

随机策略的好处:

- 自带探索。
- 可以表达多模态或不确定行为。
- 适合 policy gradient 的概率形式推导。

Log-derivative Trick

核心恒等式:

$$\nabla_{\theta} p_{\theta}(x) = p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)$$

因为:

$$\nabla_{\theta} \log p_{\theta}(x) = \nabla_{\theta} p_{\theta}(x) / p_{\theta}(x)$$

这个技巧让我们可以对采样分布依赖参数的期望求梯度。

目标:

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)]$$

梯度:

$$\begin{aligned} \nabla J(\theta) &= \nabla \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)] \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[\nabla \log p_{\theta}(\tau) R(\tau)] \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[\nabla \log p_{\theta}(\tau) R(\tau)] \\ &= \mathbb{E}[\nabla \log p_{\theta}(\tau) R(\tau)] \end{aligned}$$

trajectory 概率:

$$\begin{aligned} p_{\theta}(\tau) &= p(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t) \end{aligned}$$

环境转移 P 不依赖 θ , 所以:

$$\begin{aligned} \nabla \log p_{\theta}(\tau) &= \sum_t \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

得到 policy gradient:

$$\begin{aligned} \nabla J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[\sum_t \nabla \log \pi_{\theta}(a_t | s_t) R(\tau)] \end{aligned}$$

Policy Gradient 推导阅读线

这段推导最容易觉得“公式突然变形”。可以按下面逻辑理解。

第一步, 目标函数是期望:

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)]$$

难点在于: 被求期望的轨迹分布 $p_{\theta}(\tau)$ 本身依赖策略参数 θ 。参数变了, 采样出来的轨迹分布也变了。

第二步, 把期望写成积分或求和:

$$J(\theta) = \mathbb{E}_{\tau} p_{\theta}(\tau) R(\tau)$$

由于 reward 函数本身通常不直接对策略参数求导, 梯度主要作用在轨迹概率上:

$$\nabla J(\theta) = \mathbb{E}_{\tau} \nabla p_{\theta}(\tau) R(\tau)$$

第三步, 用 log-derivative trick 把 ∇p 改写成 $p \nabla \log p$:

$$\nabla p_{\theta}(\tau) = p_{\theta}(\tau) \nabla \log p_{\theta}(\tau)$$

这样积分又变回期望:

$$\nabla J(\theta) = \mathbb{E}_{\tau} [\nabla \log p_{\theta}(\tau) R(\tau)]$$

第四步, 展开轨迹概率:

$$p_{\theta}(\tau) = p(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

这里环境初始分布和转移概率不由策略参数控制, 所以对 θ 求梯度时只剩策略项:

$$\nabla \log p_{\theta}(\tau) = \sum_t \nabla \log \pi_{\theta}(a_t | s_t)$$

最后得到:

$$\nabla J(\theta) = \mathbb{E}_{\tau} [\sum_t \nabla \log \pi_{\theta}(a_t | s_t) R(\tau)]$$

一句话记忆:

因为采样动作本身不可直接反传, 所以 policy gradient 不对 action 求导, 而是对“采到这个 action 的 log probability”求导。高 return 的轨迹会提高其中动作的 log probability, 低 return 的轨迹会降低它们。

REINFORCE

REINFORCE 是最基础的 Monte Carlo policy gradient 算法。

更新方向:

$$\theta \leftarrow \theta + \alpha \nabla \log \pi_{\theta}(a_t | s_t) G_t$$

直觉:

- 如果某个 action 后续 return 高, 就增加该 action 在该 state 下的概率。
- 如果 return 低, 就降低该 action 的概率。

算法流程:

```
initialize policy pi_theta
repeat:
    sample trajectories using pi_theta
    compute returns G_t
    update theta by sum_t grad log pi_theta(a_t|s_t) G_t
```

REINFORCE 的优点:

- 概念简单。
- 不需要 value function。
- 可以直接处理随机策略。

缺点:

- 必须等 episode 结束。
- Monte Carlo return 方差很大。
- sample efficiency 较低。

Reward-to-go: 为什么 REINFORCE 可以用 G_t

基础推导中, 每个时间步都乘整条轨迹回报 $R(\tau)$ 。但动作 a_t 不可能影响 t 之前已经发生的 reward, 所以更合理的学习信号是从当前时间开始的 reward-to-go:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

于是实际常用:

$$\nabla J(\theta) = \mathbb{E}[\sum_t \nabla \log \pi_{\theta}(a_t | s_t) G_t]$$

这样做不会丢掉由 a_t 影响的未来回报, 反而去掉了和 a_t 无关的过去 reward, 因此方差更低。

顺序上可以这样记:

```
full trajectory return R(tau)
-> reward-to-go G_t
-> G_t - baseline
-> advantage
-> actor-critic / GAE
```

Baseline

为了降低方差, 可以从 return 中减去 baseline:

$$\begin{aligned} \nabla J(\theta) \\ = \mathbb{E}[\nabla \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t))] \end{aligned}$$

常用 baseline:

$$b(s_t) = V(s_t)$$

为什么 baseline 不引入 bias?

因为:

$$\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a|s) b(s)] = 0$$

推导:

$$\begin{aligned} \sum_a \pi(a|s) \nabla \log \pi(a|s) b(s) \\ = b(s) \sum_a \pi(a|s) \nabla \log \pi(a|s) \\ = b(s) \sum_a \nabla \pi(a|s) \\ = b(s) \nabla \sum_a \pi(a|s) \\ = b(s) \nabla 1 \\ = 0 \end{aligned}$$

只要 baseline 不依赖 action, 就不会改变 policy gradient 的期望, 只会改变方差。

更直观地说, baseline 只是把“绝对分数”改成“相对分数”:

- 原来: 这次 return 是 100, 就增加动作概率。
- 加 baseline 后: 如果这个状态平均就能拿 120, 那么 100 其实低于预期, 应该降低动作概率。

这就是 advantage 的来源。

Advantage Function

Advantage 定义:

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$$

含义:

在状态 s 下, 动作 a 比当前策略的平均动作好多少。

使用 advantage 的 policy gradient:

$$\begin{aligned} \nabla J(\theta) \\ = \mathbb{E}[\nabla \log \pi_{\theta}(a_t | s_t) A_{\pi}(s_t, a_t)] \end{aligned}$$

直觉：

- 如果 $A(s,a) > 0$ ，说明动作比平均水平好，增加其概率。
- 如果 $A(s,a) < 0$ ，说明动作比平均水平差，降低其概率。

相比直接用 return，advantage 通常方差更低，学习信号更明确。

从 Baseline 到 Actor-Critic

如果 baseline 取 $V_{\pi}(s)$ ，那么：

$$G_t - V_{\pi}(s_t)$$

就是一个 advantage 的 Monte Carlo 估计。问题是 G_t 仍然要等 episode 结束且方差大。

Actor-Critic 的下一步就是：训练一个 critic 来估计 $V(s)$ 或 $Q(s,a)$ ，用 TD target 更快、更低方差地构造 advantage。也就是说：

REINFORCE：用完整 return 做策略更新
 REINFORCE+b：用 return - baseline 降低方差
 Actor-Critic：用 critic 估计 baseline / advantage
 GAE/PP0：用更稳定的 advantage 和受限策略更新

所以第 7 章和第 8/9 章不是割裂的：后面的 actor-critic、GAE、PPO 都是在让 policy gradient 的学习信号更稳。

Entropy Regularization

为了鼓励探索，常在目标中加入 entropy：

$$J(\theta) = \mathbb{E}[\text{return}] + \beta \mathbb{E}[H(\pi_{\theta}(\cdot | s))]$$

策略熵：

$$H(\pi(\cdot | s)) = - \sum_a \pi(a | s) \log \pi(a | s)$$

作用：

- 防止策略过早变得确定。
- 增强探索。
- 改善训练稳定性。

SAC 中 entropy regularization 是核心组成部分。

本章例子：为什么需要直接优化策略

例子 1：连续动作机器人

机器人手臂要输出关节力矩：


```
a = [torque_1, torque_2, torque_3]
```

如果用 DQN, 要对连续动作求 $\max_a Q(s,a)$, 这个优化很难。Policy Gradient 直接学习:

$$\pi_{\theta}(a|s)$$

或确定性策略:

$$a = \mu_{\theta}(s)$$

这样动作可以直接由策略网络输出。

例子 2: Softmax 策略更新

假设在某个状态有两个动作:

动作	当前概率	采样后 return
left	0.4	+5
right	0.6	-1

Policy Gradient 会提高产生高 return 的动作概率, 降低低 return 动作概率。

直觉:

```
采样到 left 且结果好 -> 增大 log pi(left|s)
采样到 right 且结果差 -> 减小 log pi(right|s)
```

例子 3: Baseline 的作用

如果一个游戏平均每局 return 是 100:

- 某次动作后 return 是 105, 不算特别好, 只比平均好 5。
- 另一动作后 return 是 20, 明显差。

使用 baseline 后, 更新看的是:

$$G_t - V(s)$$

这比直接用 G_t 更能判断动作相对好坏, 方差更低。

面试表达:

Policy Gradient 的核心例子是连续控制和随机策略。它不需要枚举动作, 也不需要 Q 做 argmax, 而是让高回报轨迹上的动作概率上升。

本章面试高频问题

1. Policy Gradient 适合什么场景?

适合连续动作空间、随机策略建模、需要直接优化策略分布的场景。相比 value-based 方法, 不需要对动作做 argmax 。

2. REINFORCE 的核心更新是什么?

$$\theta \leftarrow \theta + \alpha \nabla \log \pi_{\theta}(a_t | s_t) G_t$$

如果某个动作带来高 return, 就提升该动作概率; 如果 return 低, 就降低该动作概率。

3. 为什么 baseline 不改变梯度期望?

因为 baseline 不依赖 action, 而 $\mathbb{E}_{a \sim \pi} [\nabla \log \pi(a|s)] = 0$ 。所以减去 baseline 不引入 bias, 只降低 variance。

4. Advantage 的意义是什么?

Advantage 衡量某个动作相对于该状态平均动作的好坏。它比直接用 return 更能反映动作的相对贡献, 通常降低方差并稳定训练。

5. Policy Gradient 的主要缺点?

方差较大、sample efficiency 较低、训练对学习率敏感。REINFORCE 尤其需要完整 episode, 训练慢且不稳定。

本章练习

1. 手写 log-derivative trick 推导。
2. 证明 action-independent baseline 不引入 bias。
3. 解释 $Q(s,a)$ 、 $V(s)$ 、 $A(s,a)$ 的关系。
4. 对比 value-based 和 policy-based 方法。

[章节索引](#)

[← 上一章](#) [下一章 →](#)